

SupportBot AI

KI-Kundenservice mit Transparenz und Kontrolle

Blazor Web App · .NET 8 · Gemini 2.5 Flash

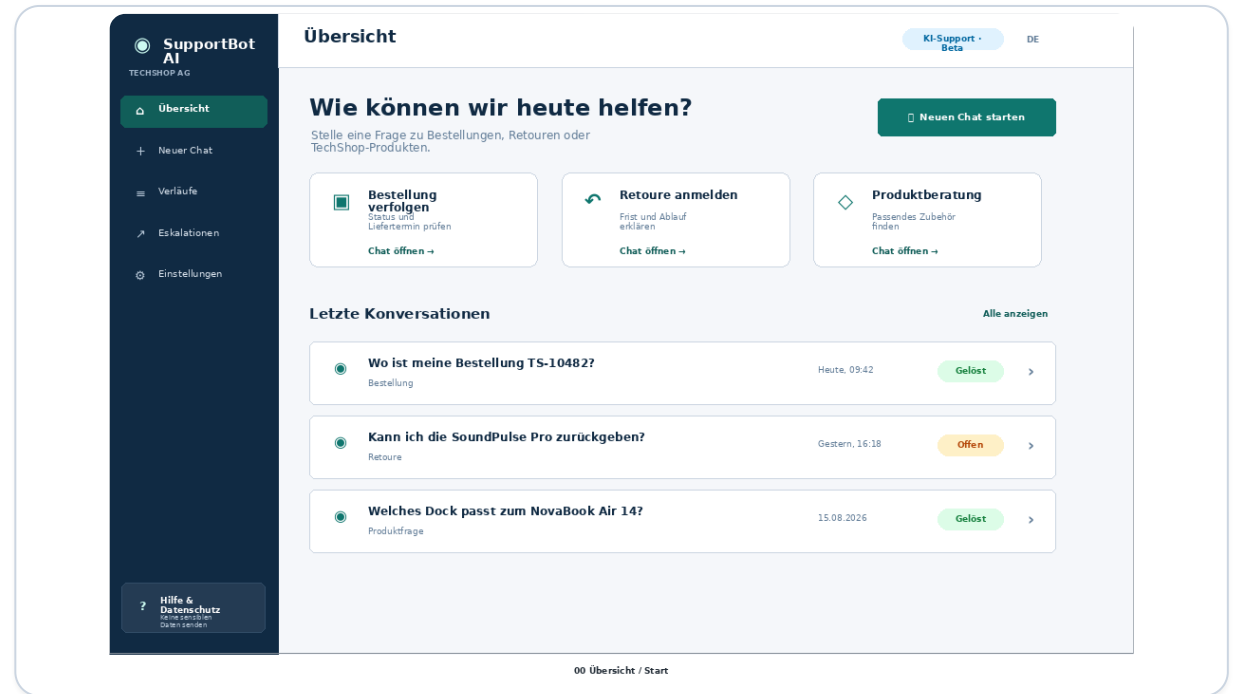
[VORNAME NACHNAME] · [KLASSE]

Das Problem: schnelle Hilfe ohne falsche Sicherheit

Kundinnen und Kunden wollen sofort wissen, wo ihre Bestellung ist. Eine generative Antwort darf dabei weder Wissen vortäuschen noch den Weg zum Support verstecken.

Produktversprechen

- schnelle Antworten
- sichtbare KI-Herkunft
- verständliche Grenzen
- menschliche Eskalation



Ein Einstieg, sechs Pflichtscreens, ein durchgängiger Hilfefluss.

Drei Personas bestimmen die Oberfläche

Lea · KI-skeptisch

Braucht klare KI-Labels, Kontext, Grenzen, Retry, Abbruch und sichtbare Eskalation.

Peter · wenig geübt

Braucht Schnellfragen, ausgeschriebene Aktionen, Formatbeispiele und verständliche Fehler.

Nora · blind

Braucht Semantik, Tastatursteuerung, aria-live, Fokusmanagement und Sprachein-/ausgabe.

Die Personas sind bewusst unterschiedlich: Vertrauen, Lernaufwand und Zugänglichkeit werden getrennt überprüfbar.

Vertrauen entsteht durch Kontrolle

Nicht durch Vermenschlichung.

TRANSPARENZ

KI-GENERIERT
Modell sichtbar
Kontext und Grenzen

STEUERBARKEIT

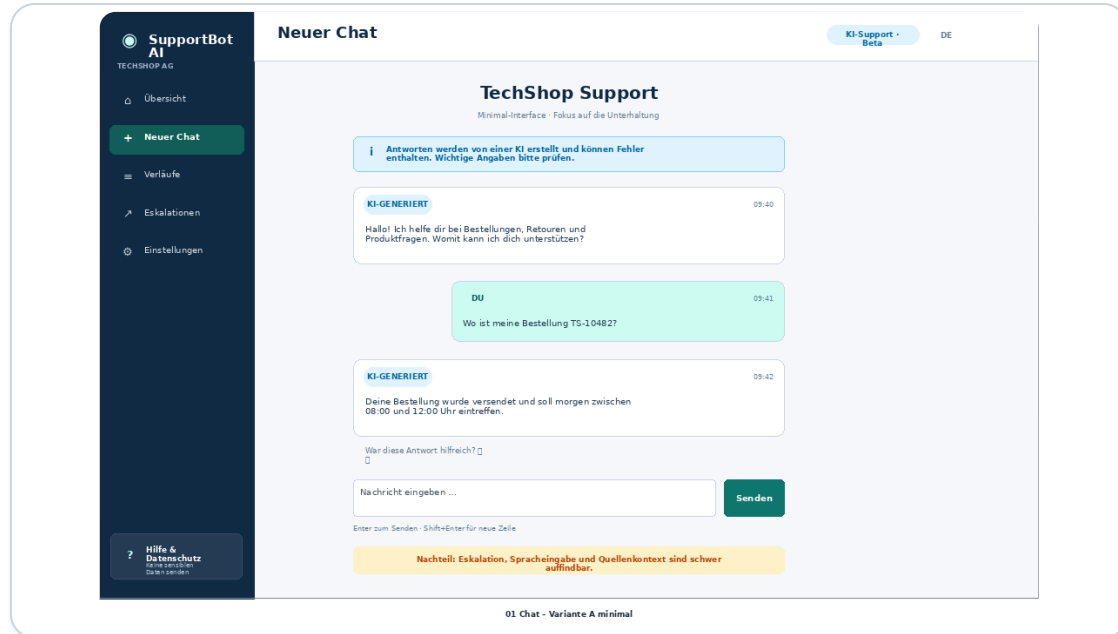
Abbrechen
Erneut versuchen
Feedback geben

FEHLERTOLERANZ

Frage bleibt erhalten
klare Wiederherstellung
Support übernehmen

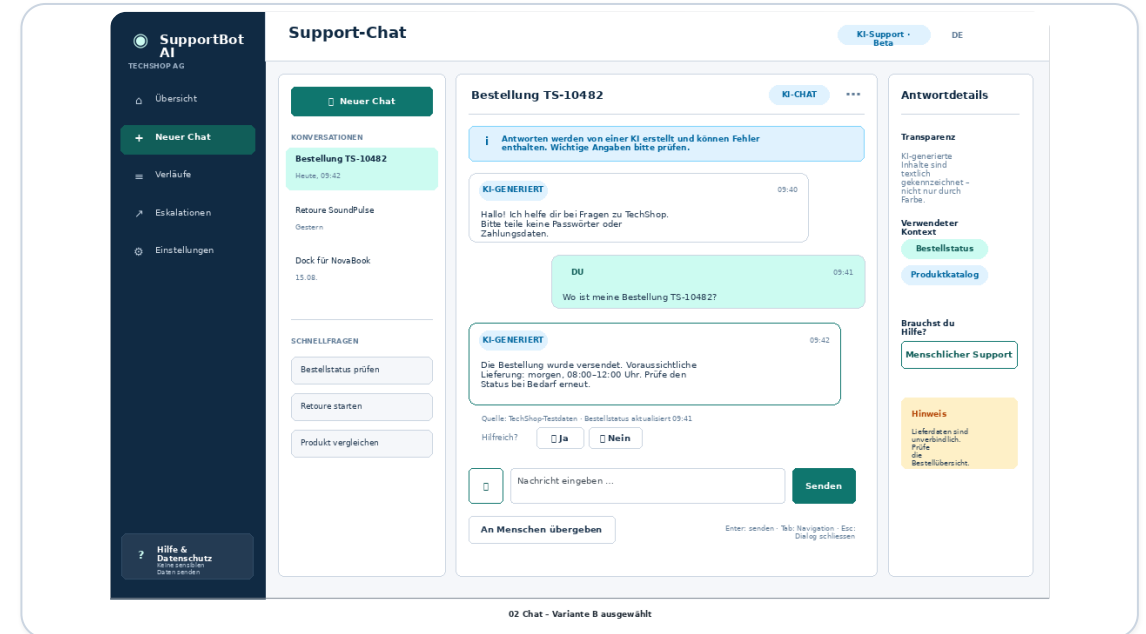
Die sieben ergonomischen Dialoggrundsätze werden dort angewendet, wo sie die Aufgabe messbar verbessern.

Zwei Chatvarianten – Variante B wird umgesetzt



A · minimal

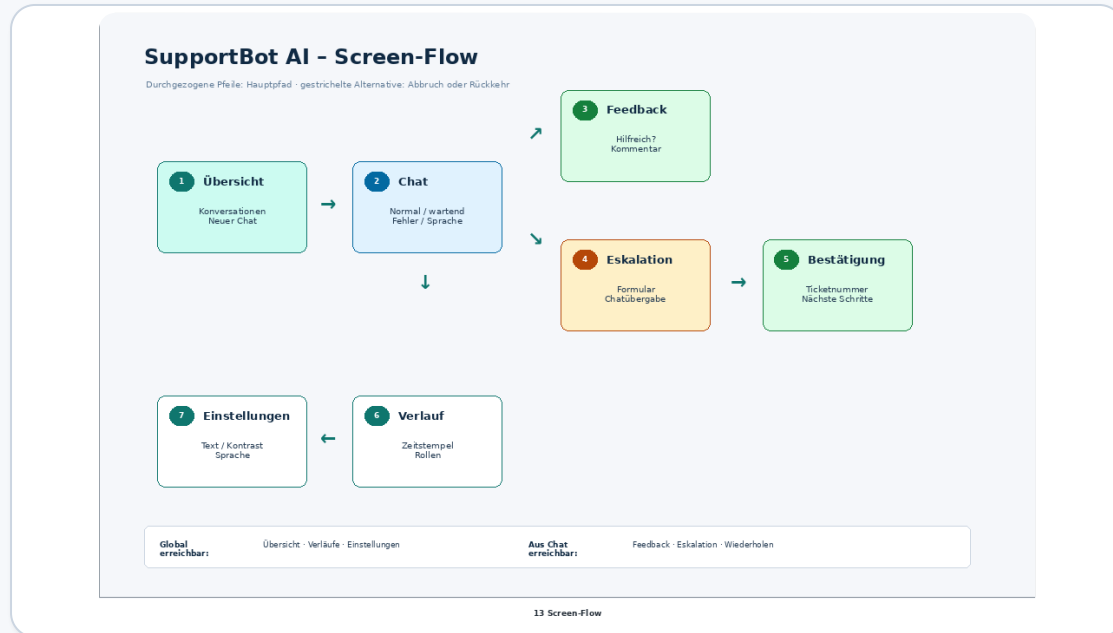
Wenig Ablenkung, aber Kontext und Kontrollmöglichkeiten sind schwerer auffindbar.



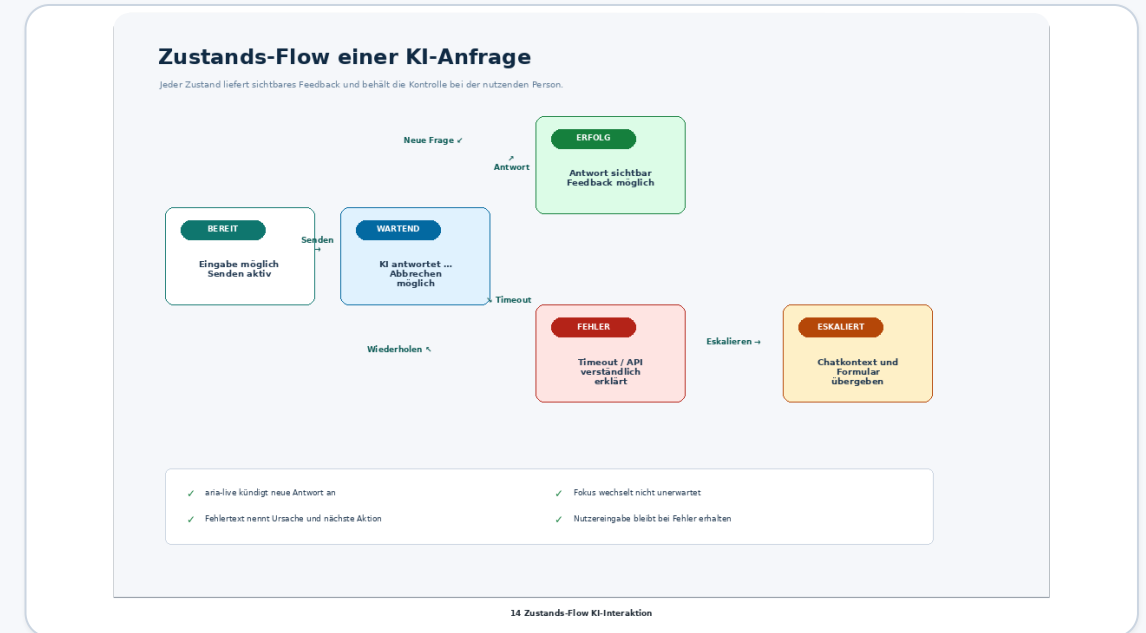
B · reichhaltig und ausgewählt

Verlauf, Schnellfragen, Modellstatus, Feedback und Eskalation bleiben direkt erreichbar.

Der Flow deckt Erfolg, Fehler und Übergabe ab



Screen-Flow

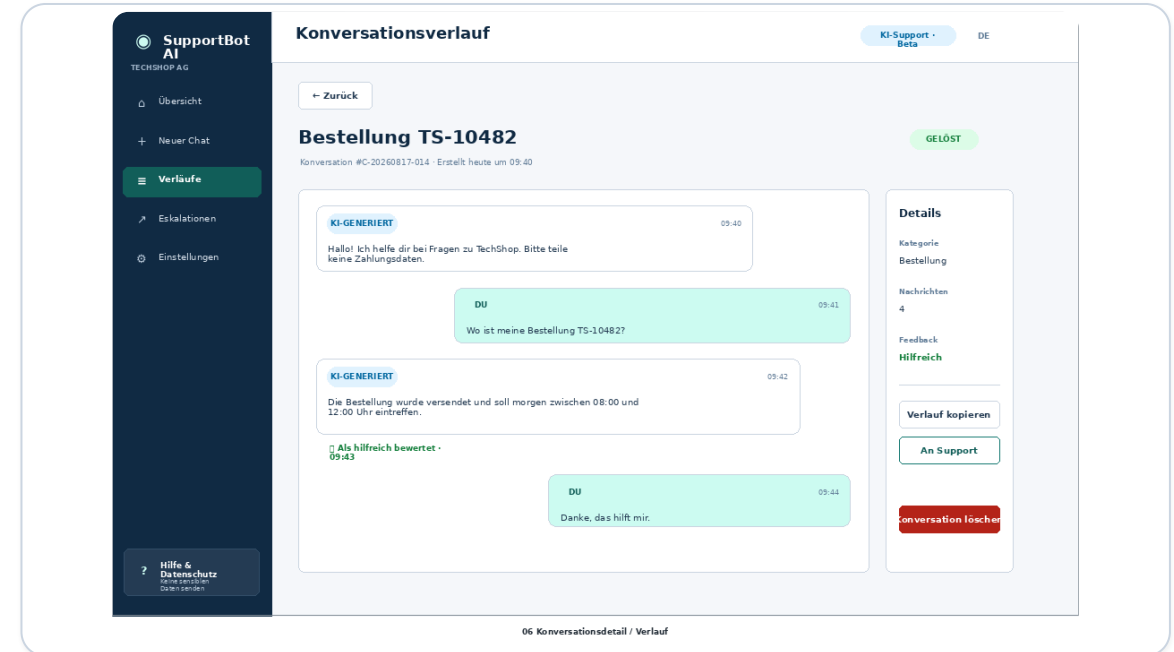


Zustands-Flow

Jeder Zustand liefert Feedback und erhält eine handlungsfähige nächste Aktion.

Live-Demo: Erfolg, Eskalation und Timeout

- 1 **Bestellung TS-10482 fragen**
- 2 **Streaming und KI-Label zeigen**
- 3 **TS-99999 an Support übergeben**
- 4 **Demo-Timeout auslösen und Retry nutzen**



Testdaten sind fiktiv; keine Passwörter oder Zahlungsdaten eingeben.

Architektur: schlank, lokal und nachvollziehbar

RAZOR UI	Chat · Verlauf · Eskalation · Feedback · Einstellungen
SERVICES	GeminiChatService · JsonAppDataStore
MODELLE	Conversation · ChatMessage · Feedback · Eskalation
BROWSER	Speech Recognition · Speech Synthesis · Fokus

.NET 8

Blazor Interactive
Server

Gemini 2.5 Flash

Google.GenAI 1.18.0

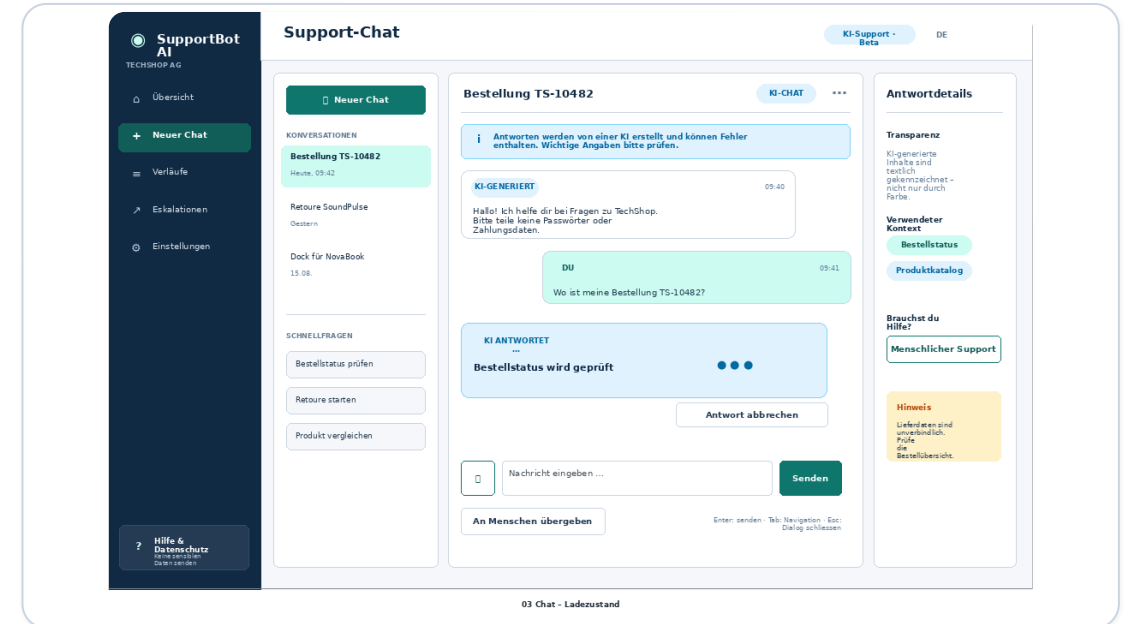
JSON-Dateien

User Secrets

API-Key und Laufzeitdaten werden nicht committet.

Kritische Komponente: Streaming bleibt kontrollierbar

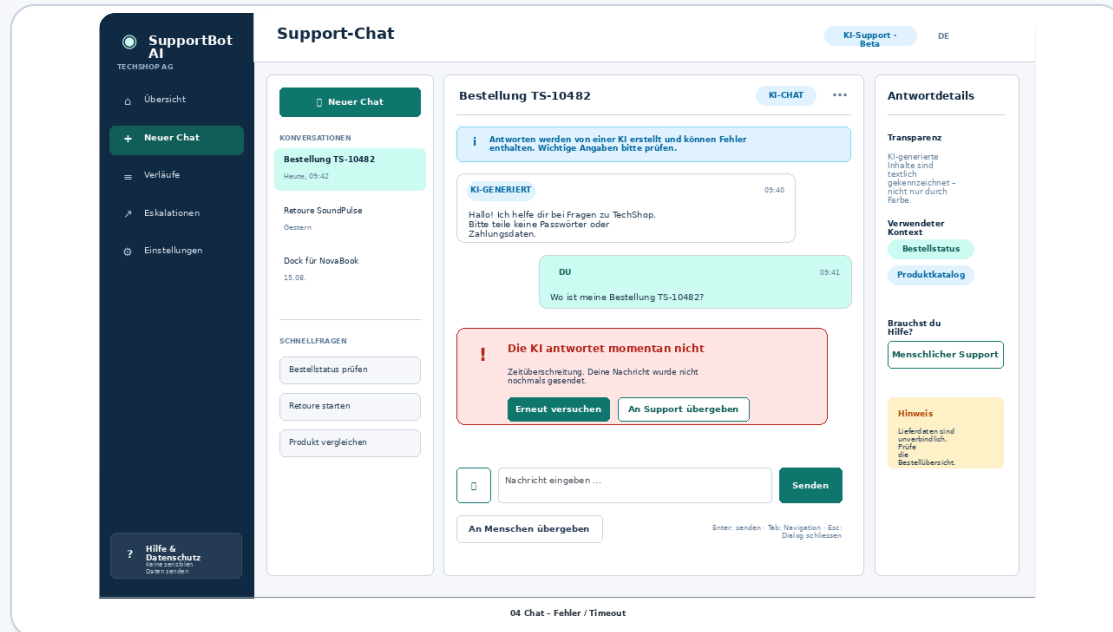
- 1 Nutzertext zuerst speichern
- 2 leere KI-Nachricht mit Streaming-Status
- 3 Textteile laufend anhängen
- 4 Timeout und Abbruch separat behandeln
- 5 Zustand immer sauber abschliessen



Invariante

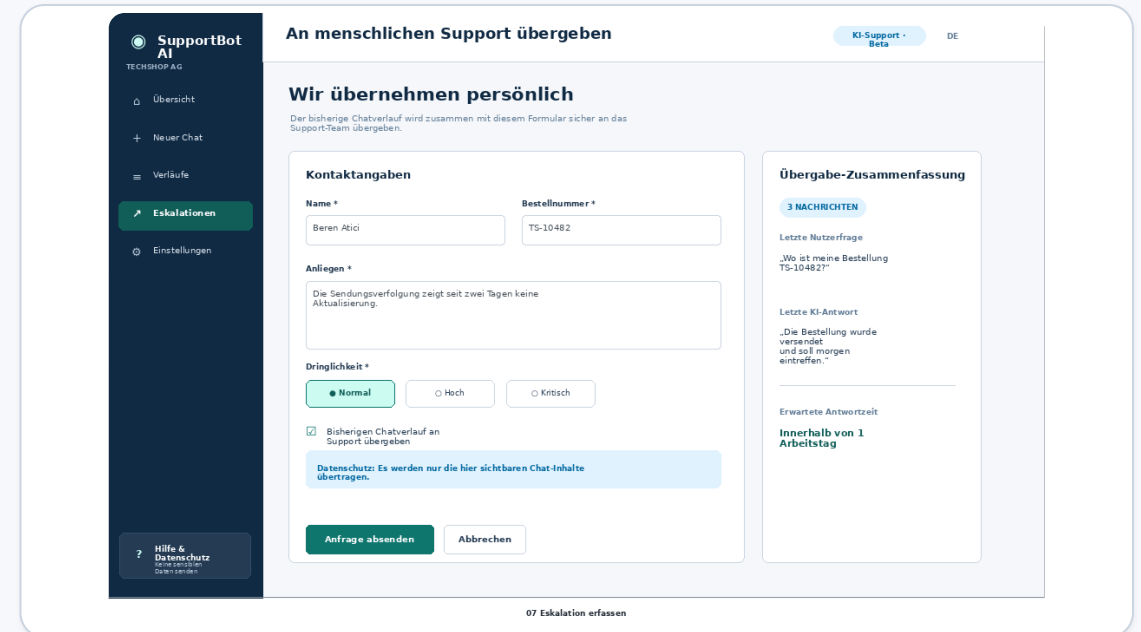
Nach Erfolg, Fehler oder Abbruch bleibt die Nutzerfrage erhalten und die Oberfläche wieder bedienbar.

Fehler ist kein Endpunkt



Timeout

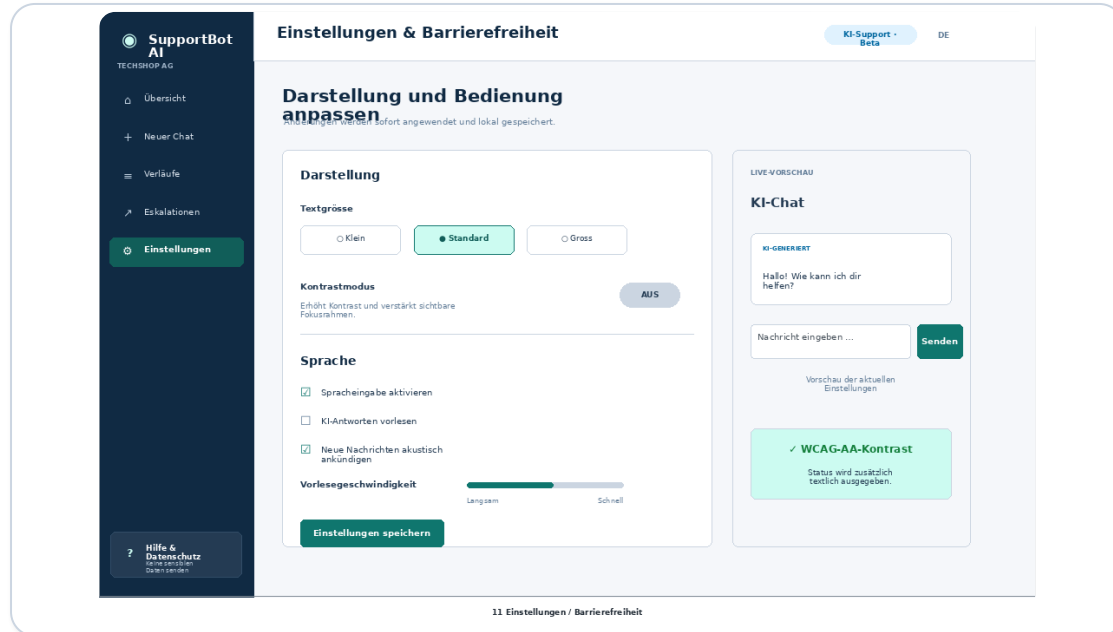
Frage bleibt erhalten · Retry · menschlicher Support



Eskalation

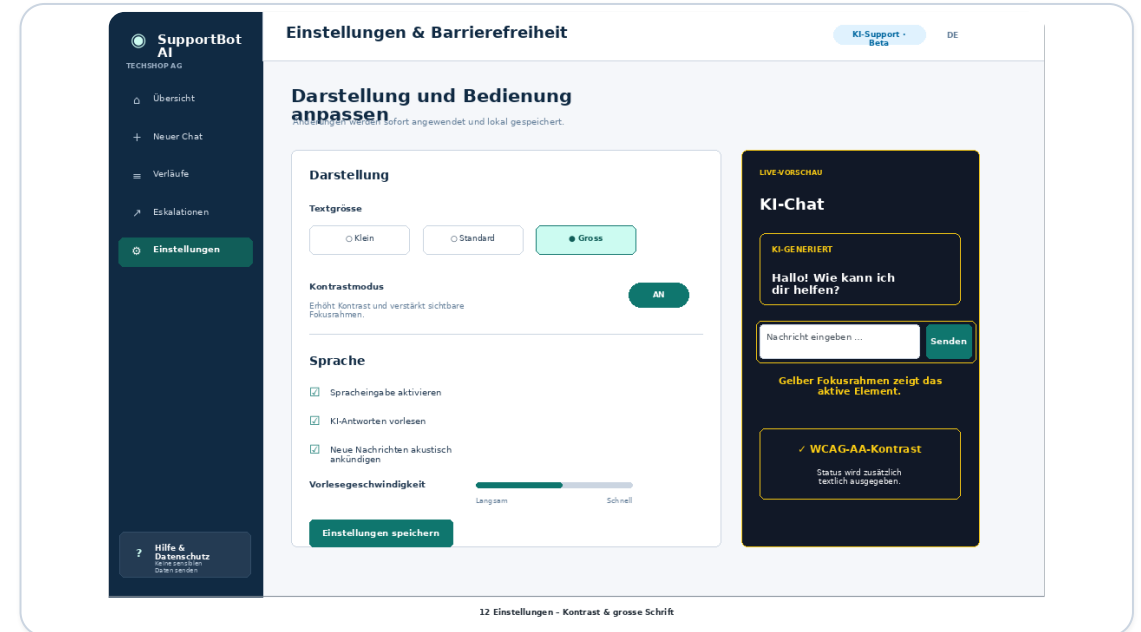
validierte Pflichtfelder · Verlauf anhängen · Bestätigung

Barrierefreiheit ist Teil des Kernflusses



Tastatur + Screenreader

Skip-Link · sichtbarer Fokus · aria-live · Fokus auf neue Antwort



Wahrnehmung + Sprache

Textgröße · hoher Kontrast · Spracheingabe · Vorlesen

Evaluation: vorbereitet, echte Daten folgen morgen

BEREITS VORBEREITET

- Walkthrough für zwei Szenarien
- Peer-Testbogen für zwei Personen
- SUS mit korrekter Auswertung
- drei Trust-Fragen
- Funktions- und WCAG-Checkliste

VOR DER ABGABE

- 1 zwei echte Personen testen
- 2 SUS + Trust auswerten
- 3 mindestens drei Befunde umsetzen
- 4 Nachprüfung dokumentieren
- 5 Lighthouse/axe-Screenshot ergänzen

Keine erfundenen Testwerte – jede Optimierung muss auf einen echten Befund zurückführbar sein.

Die wichtigste Erkenntnis

Eine gute KI-Oberfläche beantwortet nicht nur Fragen. Sie macht Unsicherheit beherrschbar.

1 Transparenz vor
Scheinpräzision

2 Fehlerzustände als
Kernfunktion

3 Barrierefreiheit von
Anfang an

Fragen zur Lösung, zum Code oder zur Live-Demo?